On this page

Top Metrics to Monitor in Reference Data Service

In this section, you will find which metrics are important to monitor in **Reference Data Service**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered the single way of monitoring the system's health.

Database Throughputâ□□

The database throughput measures the data operations' throughput for a given entity.

How to measure Database Throughputâ□□

The database throughput metric is available as `reference-data-service.entity-repository` using the `count` value. Since this metric is captured both by each operation and entity, the tags operation and entityType allow you to have a granular view for each one, respectively. For example, you can monitor the throughput of `upsert` operations for the `customer` entity.

The operations available are similar to the CRUD repository:

- **Get**: Fetch an entity's data by the primary key.
- **Delete**: Delete an entity by the primary key.
- **Upsert**: Update the entity's data or create a new entity if it doesn't exist yet.
- **List**: Retrieve a list of entities based on a set of filters.

What to monitor in Database Throughputâ□□

The database throughput metric is useful to ensure that the rate at which records are being updated by / fetched from each entity is as expected. If the number of upsert operations is below the expected, there may be a problem with the processing of reference data events. Likewise, if the number of `get` operations is not consistent with the event throughput, the system may not be enriching transactions with reference data as expected. In both scenarios, this can indicate that there is a problem somewhere in the system.

Database Latencyâ□□

The database latency metric measures the latencies per percentile (50, 95, 99 and 99.9), of the selected database operations for the specified entity.

How to measure Database Latencyâ□□

The database latency is available as `reference-data-service.entity-repository` and uses the available percentile values. Since this metric is captured both by each operation and entity, the tags `operation` and `entityType` allow you to have a granular view for each one, respectively.

The operations available are the same as mentioned above.

What to monitor in Database Latencyâ□□

The database latency metric is useful to monitor the latency of specific operations and can be very useful together with the previous throughput metric. If the throughput decreases, this metric can contribute to explain that, for example, if the latency of operations in the database degrades, there may be an issue with the database.

"Entity Not Found" Errorsâ□□

The "Entity Not Found" Errors metric measures the errors of "entity not found" type - i.e., lookups by entity key that fail because there is no record for such key.

How to measure "Entity Not Found" Errorsâ□□

The "Entity Not Found" Errors metric is available as `reference-data-service.entity-repository.entitynotfound`, using the `count` value.

What to monitor in "Entity Not Found" Errorsâ□□

The "Entity Not Found" Errors metric is useful to understand how many events miss the reference data enrichment and, when tracked over time, can hint at potential integration problems. Reference data updates are often done as a result of a Change Data Capture (CDC) process, or other data pipelines, and such processes are prone to silent failures. This metric helps identify such integration failures. The count over time is expected to be higher than zero, as there may be cases in which events from a given entity are sent before Feedzai gets any records from that same entity. Nevertheless, the the error count should be stable, and an increase is a strong indication of a process failure upstream.

Database Errorsâ□□

The Database Errors metric measures all database errors excluding the ones of type "entity not found", as previously described.

How to measure Database Errorsâ□□

The Database Errors metric is available as `reference-data-service.entity-repository.error`, using the `count` value.

What to monitor in Database Errorsâ□□

This metric shows whether there are operations failing due to issues in the database. For example, if the service receives an upsert request and the database is unresponsive, the operation fails, and this counter is incremented.

JVM metricsâ□□

Each Pulse Server and Pulse Application instance live in their own JVM process. Therefore, JVM metrics for each of these processes are available.

How to measure JVM metricsâ□□

JVM metrics are available as `letmeknow.jvm`. Youâ□□ll find many different metrics under this namespace, from GC to Memory.

What to monitor in JVM metricsâ□□

Look into at least GC and Memory metrics. These give you an idea of the healthiness of the process.